

DSA 期末机考 Cheat Paper - Markdown Only v7 Dense

使用方式：这是给 10 张 A4 正反面缩印准备的 Markdown 版；不生成 PDF。主线内容尽量往前放，不考但容量充裕的内容放附录。

用户确认不考：树状数组 BIT、线段树、倍增法、AVL；仍保留在后部附录作为保险。Python 环境按 Python 3.8：记忆化用 `@lru_cache(None)`，不要用 `@cache`。

说明：主线内容优先；这是容量增强版，增加 A/IDA/双向BFS、SPFA、割点、SCC缩点、稳健欧拉、匹配/流低优先模板、Trie/Z算法、更多DP/数论/模拟技巧。树状数组、线段树、倍增法、AVL 按用户确认不考，保留在附录。

1. P1 机考总览 + Python/OJ 输入输出

主线优先 + 附录

主线内容放 P1-P19: 输入输出、库函数、排序二分、栈队列、链表滑窗、DFS/BFS、图、最短路、MST、欧拉/Tarjan、树、堆、KMP、DP。BIT/线段树/倍增/AVL 已按不考处理，放到 P20 附录，容量够时作为保险。

做题流程

1 读题: 图/树/字符串/表达式/模拟/排序? 2 看数据范围: $n \leq 1e3$ 可 $O(n^2)$, $n \leq 1e5$ 通常 $O(n \log n)$ 或 $O(n)$ 。3 定状态/结构: 栈、队列、堆、并查集、邻接表。4 先写样例能跑的模板，再补边界: 空、单点、重复、断图、不可达。

常用开头

```
import sys, math, heapq, bisect
from collections import deque, defaultdict, Counter
from functools import lru_cache
input = sys.stdin.readline
sys.setrecursionlimit(10**7) # Python3.8: 递归DFS/树题常用
INF = 10**18
```

输入套路

```
# 1) 固定行
n, m = map(int, input().split())
a = list(map(int, input().split()))

# 2) 多组 T
t = int(input())
for _ in range(t): ...

# 3) 读到 EOF
for line in sys.stdin:
    if line.strip(): ...

# 4) 全量读取最快
data = sys.stdin.buffer.read().split()
it = iter(data)
n = int(next(it)); s = next(it).decode()

# 5) 网格
```

```

grid = [input().strip() for _ in range(n)]
# 6) 字符串编号
id = {}
def get(x):
    if x not in id: id[x] = len(id)
    return id[x]

```

输出套路

```

ans=[]
ans.append(str(x))
print('\n'.join(ans))
print(' '.join(map(str, arr)))
# 浮点: print(f'{x:.6f}')
# True/False 映射: print('YES' if ok else 'NO')

```

Python 坑

list.pop(0) 是 $O(n)$, BFS 用 deque.popleft(). 递归 DFS 大图需 setrecursionlimit; 仍可能爆栈时改迭代。Python3.8 没有 functools.cache, 用 @lru_cache(None)。字典/集合平均 $O(1)$, 排序 key 用 tuple。

2. P2 常用库函数 + 容器速查

list / str / dict / set

a.append(x); a.pop(); a[-1]; a.sort(); sorted(a)
 s.split(); ".join(list_chars); s.strip(); s[::-1]
 d.get(k, default); d.setdefault(k, [])
 set.add(x); set.remove(x); set.discard(x); x in set
 交并差: $A \cup B$, $A \cap B$, $A - B$, $A \Delta B$

collections

```

from collections import deque, defaultdict, Counter
q=deque([start]); q.append(x); q.popleft(); q.appendleft(x)
g=defaultdict(list); g[u].append(v)
cnt=Counter(arr); cnt.most_common(); cnt[x]+=1

```

heapq 小根堆/大根堆

```
import heapq
h=[]
heapq.heappush(h, x)
x = heapq.heappop(h)
heapq.heapify(a)
# 大根堆：存负数
heapq.heappush(h, -x); x = -heapq.heappop(h)
# 带 tie-breaker, 避免对象不可比
heapq.heappush(h, (dist, node))
# top k 最大：维护大小 k 的小根堆
for x in nums:
    heapq.heappush(h, x)
    if len(h)>k: heapq.heappop(h)
```

bisect 二分库

```
from bisect import bisect_left, bisect_right, insort
# 已排序数组 a
pos = bisect_left(a, x)    # 第一个 >=x
pos = bisect_right(a, x)   # 第一个 >x
cnt_x = bisect_right(a,x)-bisect_left(a,x)
# 插入保持有序：insort(a, x) # O(n) 移动
```

math / itertools

```
math.gcd(a,b); math.lcm(a,b); math.isqrt(n)
math.ceil(a/b) 可写 (a+b-1)//b, 注意负数除法
from itertools import permutations, combinations, product, accumulate
list(accumulate(a)) # 前缀和, 不含开头 0
for p in permutations(a): ...
for mask in range(1<<n): ...
```

排序 key 例子

```
# 按长度、字典序
words.sort(key=lambda s:(len(s), s))
# 按第二维降序
pairs.sort(key=lambda x:(x[0], -x[1]))
# 自定义比较尽量转 key, 不要写 cmp
```

3. P3 复杂度 + 排序 + 二分

复杂度速判

$O(1)$ 哈希查找; $O(\log n)$ 二分/堆单次; $O(n)$ 扫描; $O(n \log n)$ 排序/堆; $O(n^2)$ 双循环; $O(V+E)$ DFS/BFS; $O(E \log V)$ 堆 Dijkstra/Prim; $O(n^3)$ Floyd。

$n \leq 20$ 可状态压缩/全排列; $n \leq 500$ 可 $O(n^3)$ 勉强; $n \leq 2000$ 可 $O(n^2)$; $n \leq 1e5$ 要 $O(n \log n)$ 。

归并排序 + 逆序对

```
def merge_count(a):
    if len(a) <= 1: return a, 0
    m = len(a) // 2
    L, c1 = merge_count(a[:m]); R, c2 = merge_count(a[m:])
    i = j = cnt = 0; res = []
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            res.append(L[i]); i += 1
        else:
            res.append(R[j]); j += 1
            cnt += len(L) - i
    res += L[i:] + R[j:]
    return res, c1 + c2 + cnt
```

快速排序思想

平均 $O(n \log n)$, 最坏 $O(n^2)$ 。Python 机考一般直接 `a.sort()`。若要手写, 随机 pivot 或三路划分防重复元素退化。

二分答案框架

```
def ok(x): ...
lo, hi = 0, 10**18 # 找最小可行
while lo < hi:
    mid = (lo + hi) // 2
    if ok(mid): hi = mid
    else: lo = mid + 1
print(lo)
# 找最大可行: if ok(mid): lo = mid else hi = mid - 1, mid = (lo + hi + 1) // 2
```

左闭右闭查找

```
def bin_search(a, x):
    l, r = 0, len(a) - 1
    while l <= r:
        m = (l + r) // 2
        if a[m] == x: return m
        if a[m] < x: l = m + 1
        else: r = m - 1
    return -1
```

前缀和/差分

```
pre=[0]
for x in a: pre.append(pre[-1]+x)
# sum a[l:r] = pre[r]-pre[l]
# 一维差分: 区间 [l,r] 加 x
diff[l]+=x
if r+1<n: diff[r+1]-=x
# 最后前缀还原
```

4. P4 栈/队列/表达式/单调结构

括号匹配

```
def valid(s):
    mp={'(': ')', '[': ']', '{': '}'}; st=[]
    for ch in s:
        if ch in '([{': st.append(ch)
        elif ch in mp:
            if not st or st.pop() != mp[ch]: return False
    return not st
```

后缀表达式 RPN 求值

```
def eval_rpn(tokens):
    st=[]
    for t in tokens:
        if t not in '+-*/': st.append(int(t))
        else:
            b=st.pop(); a=st.pop()
            if t=='+': st.append(a+b)
            elif t=='-': st.append(a-b)
            elif t=='*': st.append(a*b)
            else: st.append(int(a/b)) # 向0截断
    return st[-1]
```

中缀转后缀 Shunting Yard

```
def infix_to_postfix(tokens):
    pri={' + ': 1, ' - ': 1, ' * ': 2, ' / ': 2, '^ ': 3}
    out=[]; ops=[]
    for t in tokens:
        if t.isdigit(): out.append(t)
        elif t=='(': ops.append(t)
        elif t==')':
            while ops and ops[-1]!='(': out.append(ops.pop())
            ops.pop()
        else:
```

```

        while ops and ops[-1]!='(' and pri[ops[-1]]>=pri[t]:
            out.append(ops.pop())
        ops.append(t)
    while ops: out.append(ops.pop())
    return out

```

合法出栈序列模拟

```

def can_pop_order(push_seq, pop_seq):
    st=[]; j=0
    for x in push_seq:
        st.append(x)
        while st and j<len(pop_seq) and st[-1]==pop_seq[j]:
            st.pop(); j+=1
    return j==len(pop_seq)

```

单调栈: 下一个更大元素

```

def next_greater(a):
    n=len(a); ans=[-1]*n; st=[]
    for i,x in enumerate(a):
        while st and a[st[-1]] < x:
            ans[st.pop()] = x
        st.append(i)
    return ans
# 单调栈常见：柱状图最大矩形、每日温度、左右第一个更小/更大。

```

滑动窗口最大值

```

from collections import deque
def max_window(a,k):
    q=deque(); ans=[]
    for i,x in enumerate(a):
        while q and q[0] <= i-k: q.popleft()
        while q and a[q[-1]] <= x: q.pop()
        q.append(i)
        if i>=k-1: ans.append(a[q[0]])
    return ans

```

5. P5 链表 + 双指针 + 滑动窗口

链表节点

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

```

反转链表

```
def reverse(head):
    pre=None; cur=head
    while cur:
        nxt=cur.next
        cur.next=pre
        pre=cur; cur=nxt
    return pre
```

快慢指针

```
# 找中点: slow 一步, fast 两步
slow=fast=head
while fast and fast.next:
    slow=slow.next; fast=fast.next.next
# 判环
slow=fast=head
while fast and fast.next:
    slow=slow.next; fast=fast.next.next
    if slow is fast: return True
return False
```

合并两个有序链表

```
def merge(l1,l2):
    dummy=ListNode(); cur=dummy
    while l1 and l2:
        if l1.val <= l2.val:
            cur.next=l1; l1=l1.next
        else:
            cur.next=l2; l2=l2.next
        cur=cur.next
    cur.next = l1 or l2
    return dummy.next
```

滑动窗口: 最长无重复子串

```
def longest_unique(s):
    pos={}; l=0; ans=0
    for r,ch in enumerate(s):
        if ch in pos and pos[ch] >= l:
            l = pos[ch] + 1
        pos[ch] = r
        ans = max(ans, r-l+1)
    return ans
```

双指针: 有序两数和

```
def two_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target: return l, r
        if s < target: l += 1
        else: r -= 1
    return None
```

矩阵常用方向

```
dirs4 = [(1, 0), (-1, 0), (0, 1), (0, -1)]
dirs8 = [(dx, dy) for dx in (-1, 0, 1) for dy in (-1, 0, 1) if dx or dy]
# 边界: 0 <= x < n and 0 <= y < m
```

6. P6 DFS/BFS 搜索 + 回溯

DFS 递归/迭代

```
def dfs(u):
    vis.add(u)
    for v in g[u]:
        if v not in vis: dfs(v)

# 迭代版
st = [start]; vis = {start}
while st:
    u = st.pop()
    for v in g[u]:
        if v not in vis:
            vis.add(v); st.append(v)
```

BFS 最短步数

```
from collections import deque
def bfs(s):
    dist = {s: 0}; q = deque([s])
    while q:
        u = q.popleft()
        for v in nxt(u):
            if v not in dist:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```


网格 BFS

```
def grid_bfs(starts):
    n=len(grid); m=len(grid[0])
    dist=[[-1]*m for _ in range(n)]
    q=deque()
    for x,y in starts:
        dist[x][y]=0; q.append((x,y))
    while q:
        x,y=q.popleft()
        for dx,dy in dirs4:
            nx,ny=x+dx,y+dy
            if 0<=nx<n and 0<=ny<m and grid[nx][ny]!='#' and dist[nx][ny]==-1:
                dist[nx][ny]=dist[x][y]+1
                q.append((nx,ny))
    return dist
```

多源 BFS 用途

从所有起点一起入队: 最近医院/最近岛屿/腐烂橘子/01矩阵。两岛最短桥: 先 DFS 找第一座岛, 把岛上点全部入队, 再 BFS 扩张到第二座岛。

递归深度 + Python3.8 cache

```
import sys
from functools import lru_cache
sys.setrecursionlimit(10**7)
# Python3.8 不支持 @cache, 写 @lru_cache(None)
@lru_cache(None)
def dfs(u, state):
    # state 必须可哈希: int/str/tuple/frozenset
    best = 0
    for v in nxt(u, state):
        best = max(best, 1 + dfs(v, new_state))
    return best
# 若状态含 list: dfs(tuple(arr)); 若修改状态, 用 mask 更稳。
```

记忆化搜索典型: 树/图/DP

```
# 例: 网格从 (x,y) 出发的最长递增路
@lru_cache(None)
def f(x,y):
    ans=1
    for dx,dy in dirs4:
        nx,ny=x+dx,y+dy
        if 0<=nx<n and 0<=ny<m and a[nx][ny]>a[x][y]:
            ans=max(ans,1+f(nx,ny))
    return ans
res=max(f(i,j) for i in range(n) for j in range(m))
# cache_clear(): 多组数据时 f.cache_clear() 或把函数定义放循环内。
```

A* 启发式搜索模板

```
import heapq
# h(x) 必须不高估真实距离, 常用曼哈顿距离
# 网格四方向: h=abs(x-tx)+abs(y-ty)
def astar(start, target):
    def h(p): return abs(p[0]-target[0])+abs(p[1]-target[1])
    pq=[(h(start),0,start)] # (f=g+h, g, state)
    best={start:0}
    while pq:
        f,g,u=heapq.heappop(pq)
        if u==target: return g
        if g!=best[u]: continue
        for v,c in neighbors(u):
            ng=g+c
            if ng < best.get(v, INF):
                best[v]=ng
                heapq.heappush(pq,(ng+h(v),ng,v))
    return -1
# h=0 时退化为 Dijkstra; 无权图普通最短路优先 BFS。
```

回溯模板

```
def backtrack(path, used):
    if len(path)==n:
        ans.append(path[:]); return
    for i in range(n):
        if used[i]: continue
        used[i]=True; path.append(a[i])
        backtrack(path, used)
        path.pop(); used[i]=False
```

组合/子集

```
# 子集
res=[]
def dfs(i,path):
    if i==n:
        res.append(path[:]); return
    dfs(i+1,path)
    path.append(a[i]); dfs(i+1,path); path.pop()
# 组合: for j in range(start,n): path.append; dfs(j+1)
```

7. P7 图的表示 + 连通性 + 拓扑排序

邻接表/邻接矩阵

```
# 无权图
g=[[] for _ in range(n)]
for _ in range(m):
    u,v=map(int,input().split())
    g[u].append(v); g[v].append(u) # 无向
# 带权图
g[u].append((v,w))
# 矩阵适合 n 小或稠密图
mat=[[INF]*n for _ in range(n)]
for i in range(n): mat[i][i]=0
```

连通块计数

```
vis=[False]*n
cnt=0
for i in range(n):
    if not vis[i]:
        cnt+=1
        q=deque([i]); vis[i]=True
        while q:
            u=q.popleft()
            for v in g[u]:
                if not vis[v]:
                    vis[v]=True; q.append(v)
```

有向图判环 DFS 三色

```
color=[0]*n # 0未访问 1递归栈 2完成
def dfs(u):
    color[u]=1
    for v in g[u]:
        if color[v]==1: return True
        if color[v]==0 and dfs(v): return True
    color[u]=2
    return False
has_cycle = any(color[i]==0 and dfs(i) for i in range(n))
```

拓扑排序 Kahn

```
from collections import deque
indeg=[0]*n
for u in range(n):
    for v in g[u]: indeg[v]+=1
q=deque([i for i in range(n) if indeg[i]==0])
order=[]
while q:
    u=q.popleft(); order.append(u)
    for v in g[u]:
        indeg[v]-=1
        if indeg[v]==0: q.append(v)
if len(order)<n: print('cycle')
else: print(order)
```

DAG DP

```
# 拓扑序后做动态规划
for u in order:
    for v,w in g[u]:
        dp[v] = max(dp[v], dp[u] + w)
# 课程依赖/任务调度/最长路径(无环) 常用。
```

二分图染色

```
color=[-1]*n
ok=True
for s in range(n):
    if color[s]!=-1: continue
    color[s]=0; q=deque([s])
    while q:
        u=q.popleft()
        for v in g[u]:
            if color[v]==-1:
                color[v]=color[u]^1; q.append(v)
            elif color[v]==color[u]: ok=False
```

8. P8 最短路: Dijkstra / Floyd / Bellman-Ford

Dijkstra 非负权单源最短路

```
import heapq
def dijkstra(s):
    dist=[INF]*n; dist[s]=0
    pq=[(0,s)]
    while pq:
        d,u=heapq.heappop(pq)
```

```

        if d!=dist[u]: continue
        for v,w in g[u]:
            nd=d+w
            if nd<dist[v]:
                dist[v]=nd
                heapq.heappush(pq,(nd,v))
    return dist
# 不可达 dist[v]==INF; 边权不能为负。

```

Dijkstra 还原路径

```

pre=[-1]*n
def dijkstra_path(s):
    dist=[INF]*n; dist[s]=0; pq=[(0,s)]
    while pq:
        d,u=heapq.heappop(pq)
        if d!=dist[u]: continue
        for v,w in g[u]:
            if d+w<dist[v]:
                dist[v]=d+w; pre[v]=u
                heapq.heappush(pq,(dist[v],v))
# path s->t
path=[]; x=t
while x!=-1:
    path.append(x); x=pre[x]
path=path[::-1]

```

Floyd-Warshall 多源最短路

```

dist=[row[:] for row in mat]
for k in range(n):
    for i in range(n):
        if dist[i][k]==INF: continue
        for j in range(n):
            nd=dist[i][k]+dist[k][j]
            if nd<dist[i][j]: dist[i][j]=nd
# O(n^3), n小才用; 可处理负边, 不能有负环。

```

Floyd 路径 next 矩阵

```

nxt=[[-1]*n for _ in range(n)]
for i in range(n):
    for j in range(n):
        if mat[i][j]<INF: nxt[i][j]=j
for k in range(n):
    for i in range(n):
        for j in range(n):
            if dist[i][k]+dist[k][j] < dist[i][j]:
                dist[i][j]=dist[i][k]+dist[k][j]
                nxt[i][j]=nxt[i][k]

```

```
def get_path(u,v):
    if nxt[u][v]==-1: return []
    path=[u]
    while u!=v:
        u=nxt[u][v]; path.append(u)
    return path
```

Bellman-Ford 完整函数

```
def bellman_ford(n, edges, s):
    dist=[INF]*n; pre=[-1]*n; dist[s]=0
    for _ in range(n-1):
        changed=False
        for u,v,w in edges:
            if dist[u]!=INF and dist[u]+w<dist[v]:
                dist[v]=dist[u]+w; pre[v]=u
                changed=True
        if not changed: break
    neg=False
    for u,v,w in edges:
        if dist[u]!=INF and dist[u]+w<dist[v]:
            neg=True; break
    return dist, pre, neg
# O(VE)。可处理负权边; neg=True 表示从 s 可达负环。
```

Bellman-Ford 还原路径/注意

```
# path s->t
path=[]; x=t
while x!=-1:
    path.append(x); x=pre[x]
path=path[::-1]
# 若 dist[t]==INF 表示不可达。
# 无向图的负权边 u-v 若加成两条有向边，会形成负环。
```

0-1 BFS

```
# 边权只为 0/1, 比 Dijkstra 更快
from collections import deque
dist=[INF]*n; dist[s]=0; dq=deque([s])
while dq:
    u=dq.popleft()
    for v,w in g[u]:
        if dist[u]+w < dist[v]:
            dist[v]=dist[u]+w
            if w==0: dq.appendleft(v)
            else: dq.append(v)
```

9. P9 最小生成树 + 并查集

并查集 DSU

```
class DSU:
    def __init__(self,n):
        self.fa=list(range(n)); self.sz=[1]*n
    def find(self,x):
        while self.fa[x]!=x:
            self.fa[x]=self.fa[self.fa[x]]
            x=self.fa[x]
        return x
    def union(self,x,y):
        rx,ry=self.find(x),self.find(y)
        if rx==ry: return False
        if self.sz[rx]<self.sz[ry]: rx,ry=ry,rx
        self.fa[ry]=rx; self.sz[rx]+=self.sz[ry]
        return True
```

Kruskal MST

```
def kruskal(n, edges):
    # edges: (w,u,v)
    edges.sort()
    dsu=DSU(n)
    cost=cnt=0
    for w,u,v in edges:
        if dsu.union(u,v):
            cost += w; cnt += 1
            if cnt==n-1: break
    return cost if cnt==n-1 else None
```

Prim MST 堆版

```
def prim(start=0):
    vis=[False]*n
    pq=[(0,start)]
    cost=cnt=0
    while pq and cnt<n:
        w,u=heapq.heappop(pq)
        if vis[u]: continue
        vis[u]=True; cost+=w; cnt+=1
        for v,ww in g[u]:
            if not vis[v]: heapq.heappush(pq,(ww,v))
    return cost if cnt==n else None
```

MST 判断

无向连通加权图，选 $n-1$ 条边连通所有点且总权最小。Kruskal 适合边排序；Prim 适合从点扩展。若图不连通，无 MST。

DSU 用途

1 连通块合并；2 动态连通性；3 无向图判环；4 Kruskal；5 朋友圈/集合合并。注意 DSU 不擅长删除边。

例题关键词

“修路最小花费”“连接所有城市”“最小生成树”“网络布线” -> MST。

“朋友关系”“同类合并”“是否连通” -> DSU。

“冗余连接”“加边成环” -> DSU 判环。

10. P10 欧拉路/强连通/图经典题型

无向欧拉路/回路条件

只看非零度顶点所在连通块。

欧拉回路: 所有点度数为偶数。

欧拉路径: 恰有 0 或 2 个奇度点。0 个奇度点时可从任意非零度点出发；2 个时从奇度点出发。

有向欧拉路/回路条件

只看有边顶点在弱连通意义下连通。

欧拉回路: 每点 $in=out$ 。

欧拉路径: 一个点 $out-in=1$ 为起点，一个点 $in-out=1$ 为终点，其余 $in=out$ 。

Hierholzer 输出欧拉路径

```
def euler_path(start):
    st=[start]; path=[]
    while st:
        u=st[-1]
        if g[u]:
            st.append(g[u].pop())
        else:
            path.append(st.pop())
    return path[::-1]
# 无向图要给边编号并标 used，避免反向边重复用。
```

Catenym / 单词接龙

单词作为边: 首字母 -> 尾字母。要求用完每个单词一次 = 有向欧拉路径。按字典序最小: 先按单词倒序建邻接栈，pop 出最小可用边，或排序后小心 Hierholzer。检查入出度 + 连通性。

Tarjan SCC 有向强连通

```
def tarjan_scc(g):
    n=len(g); dfn=[0]*n; low=[0]*n
    st=[]; inst=[False]*n; comp=[-1]*n; comps=[]; idx=0
    def dfs(u):
        nonlocal idx
        idx+=1; dfn[u]=low[u]=idx
        st.append(u); inst[u]=True
        for v in g[u]:
            if not dfn[v]:
                dfs(v); low[u]=min(low[u],low[v])
            elif inst[v]:
                low[u]=min(low[u],dfn[v])
        if low[u]==dfn[u]:
            cur=[]
            while True:
                x=st.pop(); inst[x]=False
                comp[x]=len(comps); cur.append(x)
                if x==u: break
            comps.append(cur)
    for i in range(n):
        if not dfn[i]: dfs(i)
    return comp, comps
# comp[u] 为 SCC 编号; comps 是每个强连通分量的点集。
```

Tarjan 桥 无向图

```
# 建图: g[u].append((v,eid)); g[v].append((u,eid))
def find_bridges(g):
    n=len(g); dfn=[0]*n; low=[0]*n; idx=0; bridges=[]
    def dfs(u, peid):
        nonlocal idx
        idx+=1; dfn[u]=low[u]=idx
        for v,eid in g[u]:
            if eid==peid: continue
            if not dfn[v]:
                dfs(v,eid); low[u]=min(low[u],low[v])
                if low[v] > dfn[u]: bridges.append((u,v))
            else:
                low[u]=min(low[u],dfn[v])
    for i in range(n):
        if not dfn[i]: dfs(i,-1)
    return bridges
# low[v] > dfn[u] => u-v 是桥。
```

图题定位

无权最短路 -> BFS; 带非负权 -> Dijkstra; 多源最短路 n小 -> Floyd; 依赖顺序 -> Topo; 连通集合 -> DSU; 用完每条边一次 -> Euler; 强连通分量 -> Kosaraju。

11. P11 二叉树遍历 + 基础递归

节点类

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val=val; self.left=left; self.right=right
```

前中后序递归

```
def preorder(root):
    if not root: return
    ans.append(root.val)
    preorder(root.left); preorder(root.right)
def inorder(root):
    if not root: return
    inorder(root.left); ans.append(root.val); inorder(root.right)
def postorder(root):
    if not root: return
    postorder(root.left); postorder(root.right); ans.append(root.val)
```

层序遍历

```
def level_order(root):
    if not root: return []
    q=deque([root]); ans=[]
    while q:
        node=q.popleft(); ans.append(node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    return ans
```

树高/节点数/叶子数

```
def height(root):
    if not root: return -1 # 按边数; 若按节点数空树为0
    return max(height(root.left), height(root.right)) + 1

def count(root):
    if not root: return 0
    return 1 + count(root.left) + count(root.right)

def leaves(root):
    if not root: return 0
    if not root.left and not root.right: return 1
    return leaves(root.left)+leaves(root.right)
```

二叉树 LCA

```
def lca(root,p,q):
    if not root or root is p or root is q: return root
    L=lca(root.left,p,q); R=lca(root.right,p,q)
    if L and R: return root
    return L or R
```

最大路径和

```
def max_path_sum(root):
    ans=-10**18
    def dfs(x):
        nonlocal ans
        if not x: return 0
        L=max(0, dfs(x.left)); R=max(0, dfs(x.right))
        ans=max(ans, x.val+L+R)
        return x.val + max(L,R)
    dfs(root); return ans
```

12. P12 建树: 前中后序/括号/字符串树

前序+中序 -> 建树

```
def build(pre, ino):
    pos={v:i for i,v in enumerate(ino)}
    def dfs(pl,pr,il,ir):
        if pl>pr: return None
        root=TreeNode(pre[pl])
        k=pos[pre[pl]]; left=k-il
        root.left=dfs(pl+1, pl+left, il, k-1)
        root.right=dfs(pl+left+1, pr, k+1, ir)
        return root
```

```
return dfs(0, len(pre)-1, 0, len(ino)-1)
```

中序+后序 -> 建树

```
def build(ino, post):
    pos={v:i for i,v in enumerate(ino)}
    def dfs(il,ir,pl,pr):
        if il>ir: return None
        root=TreeNode(post[pr])
        k=pos[post[pr]]; left=k-il
        root.left=dfs(il,k-1,pl,pl+left-1)
        root.right=dfs(k+1,ir,pl+left,pr-1)
        return root
    return dfs(0, len(ino)-1, 0, len(post)-1)
```

前中后互推技巧

前序第一个是根；后序最后一个为根；中序用根切左右子树。若有重复值，不能唯一建树，题目通常保证唯一或给编号。

括号树思路

形如 A(B,C) 或 A(B(D,E),C): 用栈维护当前父节点。读到字母建节点；'(' 表示之后读孩子；')' 切换到右孩子/下一个兄弟；')' 弹栈。具体依题面调。

表达式树

```
后缀表达式建树：
st=[]
for t in tokens:
    if t not in ops: st.append(Node(t))
    else:
        r=st.pop(); l=st.pop()
        st.append(Node(t,l,r))
root=st[-1]
# 中序输出时操作符节点加括号。
```

目录/缩进树

缩进层级 -> 栈。每行 level = 前导空格数//单位缩进；弹到父层，再加入 children。输出时 DFS，按题目要求排序或保留输入顺序。

13. P13 普通树/森林/长子兄弟 + 树形 DP

普通树邻接表

```
children=[[] for _ in range(n)]
parent=[-1]*n
for p,c in edges:
    children[p].append(c); parent[c]=p
root = next(i for i in range(n) if parent[i]==-1)
```

普通树 DFS 遍历

```
def pre(u):
    ans.append(u)
    for v in children[u]: pre(v)

def post(u):
    for v in children[u]: post(v)
    ans.append(u)
```

森林后根遍历

森林 = 多棵树。对每棵树做 postorder 后拼接。若用长子兄弟表示: firstChild 进入下一层, nextSibling 横向移动。

带度数层次序列建树

```
# 输入: A 3 B 0 C 2 ... 每个节点给 degree
from collections import deque
items = [('A',3),('B',0)]
root=items[0][0]
q=deque([(root, items[0][1])]); idx=1
children=defaultdict(list)
while q:
    u,d=q.popleft()
    for _ in range(d):
        v,deg=items[idx]; idx+=1
        children[u].append(v)
        q.append((v,deg))
```

树形 DP: 打家劫舍/宝藏树

```
# dp0: 不选u, dp1: 选u
def dfs(u):
    dp0=0; dp1=val[u]
    for v in children[u]:
        a,b=dfs(v)
        dp0 += max(a,b)
        dp1 += a
    return dp0, dp1
print(max(dfs(root)))
```

树的直径 两次 BFS/DFS

```
# 无权树: 从任意点 a 出发找最远 x, 再从 x 找最远 y, dist(x,y) 为直径。
def farthest(s):
    dist=[-1]*n; dist[s]=0; q=deque([s])
    while q:
        u=q.popleft()
        for v in g[u]:
            if dist[v]==-1:
                dist[v]=dist[u]+1; q.append(v)
    x=max(range(n), key=lambda i:dist[i])
    return x, dist[x]
```

14. P14 BST / 堆 / Huffman / 优先队列题

BST 性质

左子树 < 根 < 右子树; 中序遍历得到升序。若允许重复, 题面会规定放左/右或计数。平均查找 $O(\log n)$, 退化可 $O(n)$ 。

BST 插入/查找

```
def bst_search(root,x):
    while root and root.val!=x:
        root = root.left if x<root.val else root.right
    return root

def bst_insert(root,x):
    if not root: return TreeNode(x)
    if x<root.val: root.left=bst_insert(root.left,x)
    else: root.right=bst_insert(root.right,x)
    return root
```

堆的用途

优先队列、Top-K、合并K个有序序列、Dijkstra、Prim、Huffman、任务调度。heapq 是小根堆；复杂度 push/pop $O(\log n)$ 。

合并 K 个有序列表

```
def merge_k(lists):
    h=[]
    for i,lst in enumerate(lists):
        if lst: heapq.heappush(h,(lst[0],i,0))
    ans=[]
    while h:
        x,i,j=heapq.heappop(h); ans.append(x)
        if j+1<len(lists[i]):
            heapq.heappush(h,(lists[i][j+1],i,j+1))
    return ans
```

Huffman 合并代价

```
def huffman(weights):
    heapq.heapify(weights)
    cost=0
    while len(weights)>1:
        a=heapq.heappop(weights)
        b=heapq.heappop(weights)
        cost += a+b
        heapq.heappush(weights, a+b)
    return cost
# 每次合并两个最小权，总代价最小。
```

Top-K 高频元素

```
from collections import Counter
def topk(nums,k):
    cnt=Counter(nums)
    return [x for x,c in heapq.nlargest(k, cnt.items(), key=lambda p:p[1])]
# 也可维护大小k小根堆。
```

15. P15 字符串: KMP / 周期 / 哈希基础

KMP LPS

```
def build_lps(p):
    lps=[0]*len(p); j=0
    for i in range(1,len(p)):
        while j>0 and p[i]!=p[j]:
            j=lps[j-1]
```

```

        if p[i]==p[j]: j+=1
        lps[i]=j
    return lps

```

KMP 查找所有匹配

```

def kmp(s,p):
    if not p: return []
    lps=build_lps(p); ans=[]; j=0
    for i,ch in enumerate(s):
        while j>0 and ch!=p[j]: j=lps[j-1]
        if ch==p[j]: j+=1
        if j==len(p):
            ans.append(i-len(p)+1)
            j=lps[j-1]
    return ans
# 主串指针 i 不回退，复杂度 O(n+m)。

```

字符串周期/乘方

```

def power_count(s):
    n=len(s); lps=build_lps(s)
    p=n-lps[-1]
    if n%p==0: return n//p
    return 1
# 若 s='ababab', lps[-1]=4, p=2, count=3。

```

朴素匹配可用场景

若 n, m 很小，直接 `s.find(p)` 或 `p in s`。Python 内置 `find` 很快；但若题目要求讲算法/周期，写 KMP 更稳。

字符串处理常用

```

s.startswith(t); s.endswith(t); s.find(t)
s.replace(a,b); s.count(t)
ord('a'), chr(97)
# 判断字母数字: ch.isalpha(), ch.isdigit(), ch.isalnum()
# 大小写: s.lower(), s.upper()

```

Manacher 马拉车: 最长回文子串

```

def manacher(s):
    # O(n), 同时处理奇偶回文; 返回最长回文串和长度
    t = '^#' + '#'.join(s) + '#$'
    p = [0] * len(t)
    center = right = 0
    best_len = best_center = 0
    for i in range(1, len(t)-1):
        mir = 2 * center - i

```



```

    if i < right:
        p[i] = min(right - i, p[mir])
    while t[i+p[i]+1] == t[i-p[i]-1]:
        p[i] += 1
    if i + p[i] > right:
        center, right = i, i + p[i]
    if p[i] > best_len:
        best_len, best_center = p[i], i
    start = (best_center - best_len) // 2
    return s[start:start+best_len], best_len
# 原串中最长长度 = max(p); start=(center-len)//2。

```

滚动哈希提醒

固定长子串去重可直接 `set(s[i:i+k])`; 若 n 很大且 k 小, 用滚动整数/位运算。哈希有冲突, 机考一般避免复杂双哈希。

16. P16 动态规划/贪心/常见算法套路

DP 基本想法

状态: `dp[i]` 表示什么? 转移: 从哪些旧状态来? 初始化: 空/第一个? 答案: `max/min/sum`? 顺序: 依赖谁先算谁。

一维最大子段和 Kadane

```

best=cur=a[0]
for x in a[1:]:
    cur=max(x, cur+x)
    best=max(best, cur)
print(best)

```

LIS 最长上升子序列 $O(n \log n)$

```

from bisect import bisect_left
d=[]
for x in a:
    i=bisect_left(d,x)
    if i==len(d): d.append(x)
    else: d[i]=x
print(len(d))
# d[i] 是长度 i+1 的上升子序列最小结尾。

```

0/1 背包

```
dp=[0]*(w+1)
for w,val in items:
    for cap in range(w,w-1,-1):
        dp[cap]=max(dp[cap], dp[cap-w]+val)
print(dp[w])
```

完全背包

```
dp=[0]*(w+1)
for w,val in items:
    for cap in range(w,w+1):
        dp[cap]=max(dp[cap], dp[cap-w]+val)
```

贪心常见

区间调度: 按结束时间排序, 能选就选。

最少会议室: 按开始排序 + 小根堆维护结束时间。

合并区间: 按左端点排序, 能合并就扩右端。

Huffman: 每次取两个最小合并。

合并区间

```
intervals.sort()
res=[]
for l,r in intervals:
    if not res or l>res[-1][1]: res.append([l,r])
    else: res[-1][1]=max(res[-1][1],r)
```

17. P17 位运算/集合状态压缩/数学小工具

位运算基础

$x \& 1$ 判断奇偶; $x \gg= 1$ 右移; $x \ll k$ 乘 2^k 。

$\text{lowbit} = x \& -x$ # 虽然 BIT 不考, 但位技巧常见

判断 2 的幂: $x > 0$ and $x \& (x-1) == 0$

bit_count : $x.\text{bit_count}()$; bit_length : $x.\text{bit_length}()$

状态压缩枚举

```
n=10
for mask in range(1<<n):
    for i in range(n):
        if mask>>i & 1:
            ...
# 加入 i: mask | (1<<i)
# 删除 i: mask & ~(1<<i)
# 翻转 i: mask ^ (1<<i)
```

枚举子集

```
sub = mask
while sub:
    # sub 是 mask 的非空子集
    sub = (sub-1) & mask
# 包括空集可在循环后处理 sub=0
```

GCD/质数

```
import math
math.gcd(a,b)
def is_prime(x):
    if x<2: return False
    d=2
    while d*d<=x:
        if x%d==0: return False
        d+=1
    return True
```

快速幂

```
def qpow(a,b,mod=None):
    res=1
    while b:
        if b&1: res=res*a if mod is None else res*a%mod
        a=a*a if mod is None else a*a%mod
        b//=2
    return res
# Python内置: pow(a,b,mod)
```

坐标压缩

```
vals=sorted(set(a))
rank={x:i for i,x in enumerate(vals)}
compressed=[rank[x] for x in a]
# 用于值域大但只关心相对大小。
```

组合数小工具

```
# Python 3.8+  
math.comb(n,k); math.perm(n,k)  
# 取模组合数需预处理阶乘/逆元，除非题目给小数据。
```

18. P18 经典例题: 题意 -> 模板

表达式类

括号匹配 -> 栈。

后缀表达式求值 -> 栈。

中缀转后缀 -> Shunting Yard。

表达式树 -> 后缀建树，操作符节点弹两个孩子。

树类

前中/中后序建树 -> 根切中序。

求树深/叶子/路径和 -> DFS。

层序输出/最短层数 -> BFS。

森林后根遍历 -> 建 children 后 postorder。

Huffman 编码/最小合并代价 -> heapq。

图类

迷宫最短路/骑士/网格扩散 -> BFS。

岛屿数量 -> DFS/BFS 连通块。

课程安排/依赖 -> 拓扑排序。

城市最短路非负权 -> Dijkstra。

任意两点最短路 n 小 -> Floyd。

修路最小费用 -> MST。

单词接龙用完所有词 -> 欧拉路径。

字符串类

查找模式串所有出现位置 -> KMP。

字符串乘方/循环节 -> KMP lps[-1]。

单词统计 -> Counter/dict。

前缀匹配若 Trie 不考，可用排序/startswith/字典集合。

模拟类

先把状态定义清楚: 位置、方向、时间、队列、栈。复杂模拟优先写函数 move/turn/check，别把所有逻辑挤主循环。

排序类

要求第 k 大 $\rightarrow$ `heapq.nlargest` 或小根堆维护 k 个。

逆序对 $\rightarrow$ 归并计数。

区间合并/安排 $\rightarrow$ 排序 + 贪心。

按多关键字排序 $\rightarrow$ `key tuple`。

边界测试清单

$n=0/1$ ；空图/孤立点；重复边/自环；不可达；负数；全相等；已经有序/逆序；多组输入；输出末尾空格；递归深度。

19. P19 常见 bug 修正表 + 调试

索引/闭区间

Python 切片 `[l:r)` 不含 r 。题目若给 1-based，读入后通常 $u-=1$ ； $v-=1$ 。区间 `[l,r]` 的前缀和是 `pre[r+1]-pre[l]`。

图建边

无向边要加两次；有向边只加一次。入度/出度别反。带权边 `g[u].append((v,w))`，遍历写 `for v,w in g[u]`。

DFS visited 时机

BFS/DFS 入队/入栈时就标记，避免重复入队导致爆炸。回溯题则需要撤销 `used/path`。

Dijkstra 坑

有负边不能用。heap 中旧状态要 `if d!=dist[u]: continue`。不可达 INF 输出按题意处理。

拓扑排序坑

队列初始是所有入度 0 点。结果长度 $< n$ 说明有环。若要求字典序最小拓扑序，把 `deque` 换成 `heapq`。

KMP 坑

失配时 `while j>0 and s[i]!=p[j]: j=ps[j-1]`。匹配完整后 `j=ps[j-1]`，否则会漏掉重叠匹配。

树建造坑

`TreeNode(pre[pl])` 不是 `preorder(pl)`。中序搜索不要每次 `for`，预处理 `pos` 字典。空子树判断 `pl>pr` 或 `il>ir`。

本地调试技巧

先打印中间结构: `g, indeg, order, dist`。小样例手算一遍。递归输出进入/退出节点可查遍历顺序。提交前删调试 `print`。

20. P20 附录: 不考/低优先内容 + 最终索引

代码索引

P1 输入输出; P2 heapq/bisect/Counter; P3 归并逆序对/二分; P4 栈表达式/单调队列; P5 链表/滑窗; P6 DFS/BFS/回溯/cache/A*; P7 图连通/拓扑; P8 Dijkstra/Floyd/Bellman-Ford; P9 MST/DSU; P10 欧拉/Tarjan; P11-13 树; P14 堆/Huffman/BST; P15 KMP/Manacher/Manacher; P16 DP/贪心; P17 位运算; P18 题型; P19 bug; P20 附录。

附录说明

BIT、线段树、倍增法、AVL 按当前确认不考，放最后。考场优先用主线模板；除非题面非常直白要求区间动态维护/LCA/平衡树，否则不要先写这些。Tarjan 图算法和 Bellman-Ford 已放 P10/P8。

BIT 树状数组 optional

```
class BIT:
    def __init__(self,n):
        self.n=n; self.t=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n:
            self.t[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0:
            s+=self.t[i]; i-=i&-i
        return s
    def range_sum(self,l,r):
        return self.sum(r)-self.sum(l-1)
# 1-indexed; 逆序对常配离散化 rank。
```

线段树 optional: 区间和

```
class SegTree:
    def __init__(self,a):
        n=1
        while n<len(a): n*=2
        self.n=n; self.t=[0]*(2*n)
        for i,x in enumerate(a): self.t[n+i]=x
        for i in range(n-1,0,-1): self.t[i]=self.t[i*2]+self.t[i*2+1]
    def set(self,i,x):
        i+=self.n; self.t[i]=x
        while i>1:
            i//=2; self.t[i]=self.t[i*2]+self.t[i*2+1]
    def query(self,l,r): # [l,r)
        l+=self.n; r+=self.n; s=0
        while l<r:
            if l&1: s+=self.t[l]; l+=1
            if r&1: r-=1; s+=self.t[r]
            l//=2; r//=2
```

```
return s
```

倍增 LCA optional

```
LOG=(n).bit_length()
up=[[-1]*n for _ in range(LOG)]
dep=[0]*n
def dfs(u,p):
    up[0][u]=p
    for v in g[u]:
        if v!=p:
            dep[v]=dep[u]+1; dfs(v,u)
for k in range(1,LOG):
    for v in range(n):
        if up[k-1][v]!=-1:
            up[k][v]=up[k-1][up[k-1][v]]
def lca(a,b):
    if dep[a]<dep[b]: a,b=b,a
    d=dep[a]-dep[b]
    for k in range(LOG):
        if d>>k&1: a=up[k][a]
    if a==b: return a
    for k in range(LOG-1,-1,-1):
        if up[k][a]!=up[k][b]: a=up[k][a]; b=up[k][b]
    return up[0][a]
```

AVL optional 只记旋转

BST + height 平衡。bf = height(left)-height(right)。

LL: 右旋 root; RR: 左旋 root; LR: left 先左旋, 再 root 右旋; RL: right 先右旋, 再 root 左旋。

right_rotate(y): x=y.left; T2=x.right; x.right=y; y.left=T2; 更新 y,x 高度; return x。

left_rotate(x): y=x.right; T2=y.left; y.left=x; x.right=T2; 更新 x,y 高度; return y。

考试策略/心态

先拿简单分: 输入输出、模拟、排序、BFS。卡 15 分钟没进展就写暴力/部分分。最后查边界: 空、单点、重复、断图、不可达、输出格式。机考不是背所有高级算法, 是识别题型 + 稳定写模板。

21. P21 Python3.8 环境细节 + 超快输入输出补充

Python3.8 可用/不可用

可用: f-string, walrus :=, math.isqrt, math.comb, math.perm, functools.lru_cache, dataclasses。

不可用: functools.cache(Python3.9+), match-case(Python3.10+), str.removeprefix(Python3.9+)。考试写 @lru_cache(None)。

极快 Scanner

```
import sys
class FastScanner:
    def __init__(self):
        self.data=sys.stdin.buffer.read().split(); self.i=0
    def next(self):
        x=self.data[self.i]; self.i+=1; return x
    def int(self): return int(self.next())
    def str(self): return self.next().decode()
fs=FastScanner()
n=fs.int(); arr=[fs.int() for _ in range(n)]
# 全是整数时最快，但混合行格式/含空格字符串不要乱用。
```

按哨兵读入

```
# 读到单独一行 . / 0 0 / EOF
for line in sys.stdin:
    line=line.strip()
    if line=='.': break
    ...
while True:
    a,b=map(int,input().split())
    if a==0 and b==0: break
    ...
```

递归与栈安全

```
import sys
sys.setrecursionlimit(10**7)
# 深链树/大图递归仍可能 RuntimeError/段错误，改迭代。
# 递归函数局部变量太多也更容易爆；大数组放外层。
```

lru_cache 多组数据坑

```
from functools import lru_cache
@lru_cache(None)
def f(x,y): ...
# 多组测试且 f 依赖全局 grid/n/m 时：每组后 f.cache_clear()
# 或把 @lru_cache 函数定义在每组循环内部。
```

常用排序工具

```
from operator import itemgetter
pairs.sort(key=itemgetter(1))
from functools import cmp_to_key
# cmp(a,b)<0 表示 a 排前；能写 key 就别写 cmp。
a.sort(key=cmp_to_key(cmp))
```


输出大数据

```
out=[]
for x in ans: out.append(str(x))
sys.stdout.write('
'.join(out))
# 不要循环里大量 print: 会慢。
```

22. P22 启发式搜索进阶: A* / IDA* / 双向 BFS / 状压 BFS

A* 使用判断

适合: 有明确目标 target, 边权非负, 有可采纳启发 $h(\text{state}) \leq \text{真实剩余代价}$ 。
网格最短路常用曼哈顿距离; $h=0$ 退化成 Dijkstra。若无权图且状态少, BFS 更简单。

IDA* 模板

```
# 迭代加深 + 启发式剪枝: 常见于八数码/拼图类
FOUND = object()
def ida_star(start):
    bound = h(start)
    path = [start]
    while True:
        t = search(path, 0, bound)
        if t is FOUND: return path[:]
        if t == INF: return None
        bound = t

def search(path, g, bound):
    u = path[-1]
    f = g + h(u)
    if f > bound: return f
    if is_goal(u): return FOUND
    mn = INF
    for v, c in neighbors(u):
        if v in path: continue
        path.append(v)
        t = search(path, g+c, bound)
        if t is FOUND: return FOUND
        mn = min(mn, t)
    path.pop()
    return mn
```

双向 BFS

```
def bidir_bfs(s, t):
    if s == t: return 0
    A = {s}; B = {t}; da = {s:0}; db = {t:0}
    while A and B:
```

```

if len(A)>len(B): A,B=B,A; da,db=db,da
NA=set()
for u in A:
    for v in nxt(u):
        if v in db: return da[u]+1+db[v]
        if v not in da:
            da[v]=da[u]+1; NA.add(v)
A=NA
return -1

```

状态图巨大但目标明确时很好用。

状压 BFS

```

# 例: state=(pos, mask), 访问所有点最短路
q=deque([(start, 1<<start)])
dist={(start,1<<start):0}
while q:
    u,mask=q.popleft()
    if mask == (1<<n)-1: print(dist[(u,mask)]); break
    for v in g[u]:
        nm = mask | (1<<v)
        st=(v,nm)
        if st not in dist:
            dist[st]=dist[(u,mask)]+1
            q.append(st)

```

BFS 路径还原

```

pre={start: None}; q=deque([start])
while q:
    u=q.popleft()
    if u==target: break
    for v in nxt(u):
        if v not in pre:
            pre[v]=u; q.append(v)
path=[]; x=target
while x is not None:
    path.append(x); x=pre[x]
path.reverse()

```

剪枝常用

1 当前代价 >= best 直接停。2 剩余理论最优也不可能超过答案时停。3 搜索顺序先走更可能成功的分支。4 visited/ 状态压缩避免重复。5 对称状态只搜一种。

23. P23 图算法补充: SPFA / 割点 / SCC 缩点 / 字典序拓扑

SPFA 队列优化 Bellman-Ford

```
from collections import deque
def spfa(s):
    dist=[INF]*n; inq=[False]*n; cnt=[0]*n
    dist[s]=0; q=deque([s]); inq[s]=True
    while q:
        u=q.popleft(); inq[u]=False
        for v,w in g[u]:
            if dist[u]!=INF and dist[u]+w<dist[v]:
                dist[v]=dist[u]+w
                if not inq[v]:
                    q.append(v); inq[v]=True; cnt[v]+=1
                    if cnt[v]>=n: return None # 可能负环
    return dist
# 最坏 O(VE), 一般不如 Dijkstra 稳; 有负边且数据友好可用。
```

Tarjan 割点 articulation point

```
def articulation_points(g):
    n=len(g); dfn=[0]*n; low=[0]*n; idx=0; cut=[False]*n
    def dfs(u, parent):
        nonlocal idx
        idx+=1; dfn[u]=low[u]=idx; child=0
        for v in g[u]:
            if not dfn[v]:
                child+=1; dfs(v,u); low[u]=min(low[u],low[v])
                if parent!=-1 and low[v] >= dfn[u]: cut[u]=True
            elif v!=parent:
                low[u]=min(low[u],dfn[v])
        if parent==-1 and child>=2: cut[u]=True
    for i in range(n):
        if not dfn[i]: dfs(i,-1)
    return [i for i,x in enumerate(cut) if x]
# 删除该点会增加连通块数量。
```

SCC 缩点 DAG

```
comp, comps = tarjan_scc(g)
c=len(comps)
dag=[set() for _ in range(c)]
indeg=[0]*c
for u in range(n):
    for v in g[u]:
        a,b=comp[u],comp[v]
        if a!=b and b not in dag[a]:
            dag[a].add(b); indeg[b]+=1
# 缩点后一定是 DAG, 可做拓扑 DP。
```

字典序最小拓扑序

```
import heapq
h=[i for i in range(n) if indeg[i]==0]
heapq.heapify(h); order=[]
while h:
    u=heapq.heappop(h); order.append(u)
    for v in g[u]:
        indeg[v]-=1
        if indeg[v]==0: heapq.heappush(h,v)
# 若节点是字符串, heap 存字符串或编号按题意排序。
```

Kosaraju SCC 简版

```
def kosaraju(g, rg):
    n=len(g); vis=[False]*n; order=[]
    def dfs1(u):
        vis[u]=True
        for v in g[u]:
            if not vis[v]: dfs1(v)
        order.append(u)
    def dfs2(u,c):
        comp[u]=c
        for v in rg[u]:
            if comp[v]==-1: dfs2(v,c)
    for i in range(n):
        if not vis[i]: dfs1(i)
    comp=[-1]*n; c=0
    for u in reversed(order):
        if comp[u]==-1:
            dfs2(u,c); c+=1
    return comp,c
```

24. P24 欧拉路径稳健实现 + 图建模例题

无向欧拉路径 Hierholzer with edge id

```
def euler_undirected(n, edge_list, start):
    g=[] for _ in range(n)
    for eid,(u,v) in enumerate(edge_list):
        g[u].append((v,eid)); g[v].append((u,eid))
    used=[False]*len(edge_list); st=[start]; path=[]
    while st:
        u=st[-1]
        while g[u] and used[g[u][-1][1]]:
            g[u].pop()
        if g[u]:
            v,eid=g[u].pop()
            used[eid]=True; st.append(v)
```

```

    else:
        path.append(st.pop())
    return path[::-1]
# len(path)==m+1 才说明所有边用完。

```

有向欧拉路径选起点

```

def choose_start(n, indeg, outdeg):
    start=-1; plus=minus=0
    for i in range(n):
        d=outdeg[i]-indeg[i]
        if d==1: plus+=1; start=i
        elif d==-1: minus+=1
        elif d!=0: return -1
    if (plus,minus)==(0,0):
        return next((i for i in range(n) if outdeg[i]>0),0)
    if (plus,minus)==(1,1): return start
    return -1

```

欧拉连通性检查

```

# 有向欧拉通常检查所有有边点在弱连通意义下连通。
# 建无向版 ug, 把 indeg+outdeg>0 的点拿一个做 BFS。
# BFS 后所有 degree>0 点都应 visited。

```

单词链 Catenym 模板思路

单词按边建: $u = \text{ord}(\text{word}[0]) - 97$, $v = \text{ord}(\text{word}[-1]) - 97$ 。

要输出字典序最小链: `words.sort(reverse=True)`, `g[u].append((v,word))`, Hierholzer pop 最小词。

路径是边序, 回溯时 append word, 最后 reverse。

课程先修/任务调度

边 `prereq -> course`; 入度为0表示当前能学。若要每次编号最小课程, 用小根堆拓扑。若有环, 则无法完成。

网格转图

每个格子可映射 $\text{id} = x \cdot m + y$; 但网格 BFS 一般直接用 (x,y) 更清楚。

若要 DSU 连通: $\text{id} = \lambda x, y: x \cdot m + y$ 。

25. P25 匹配/流/高级图低优先模板

二分图最大匹配 Kuhn

```

# 左部 0..L-1, 右部 0..R-1, g[u] 是右部邻居
match=[-1]*R
def dfs(u,vis):
    for v in g[u]:
        if vis[v]: continue

```

```

vis[v]=True
if match[v]==-1 or dfs(match[v],vis):
    match[v]=u; return True
return False
ans=0
for u in range(L):
    if dfs(u,[False]*R): ans+=1
# O(VE), 适合中小数据。

```

Dinic 最大流 optional

```

class Dinic:
    def __init__(self,n): self.n=n; self.g=[[] for _ in range(n)]
    def add(self,u,v,c):
        self.g[u].append([v,c,len(self.g[v])])
        self.g[v].append([u,0,len(self.g[u])-1])
    def maxflow(self,s,t):
        flow=0
        while True:
            level=[-1]*self.n; q=deque([s]); level[s]=0
            while q:
                u=q.popleft()
                for v,c,rev in self.g[u]:
                    if c>0 and level[v]<0: level[v]=level[u]+1; q.append(v)
            if level[t]<0: return flow
            it=[0]*self.n
            def dfs(u,f):
                if u==t: return f
                for i in range(it[u],len(self.g[u])):
                    it[u]=i; v,c,rev=self.g[u][i]
                    if c>0 and level[u]+1==level[v]:
                        ret=dfs(v,min(f,c))
                        if ret:
                            self.g[u][i][1]-=ret; self.g[v][rev][1]+=ret; return ret
                return 0
            while True:
                f=dfs(s,10**18)
                if not f: break
            flow+=f
# 若没学过, 放最后; 匹配/割/流量关键词才考虑。

```

最小割直觉

最大流 = 最小割容量。题面出现“最少删除边使 s,t 不连通”“最大运输量”可能是流。一般期末机考不优先写。

差分约束提示

形如 $x_v \leq x_u + w$, 可建边 $u \rightarrow v$ 权 w , 求最短路; 有负环表示无解。属于低优先高级内容。

26. P26 字符串补充: Trie / Z算法 / 回文DP / 滚动整数

Trie 前缀树 optional

```
class Trie:
    def __init__(self): self.ch={}; self.end=False
root=Trie()
def insert(s):
    p=root
    for c in s:
        if c not in p.ch: p.ch[c]=Trie()
        p=p.ch[c]
    p.end=True
def search(s):
    p=root
    for c in s:
        if c not in p.ch: return False
        p=p.ch[c]
    return p.end
```

前缀查询/字典树; 若不考, 排序+startswith 有时够用。

Z algorithm

```
def z_function(s):
    n=len(s); z=[0]*n; l=r=0
    for i in range(1,n):
        if i<=r: z[i]=min(r-i+1,z[i-l])
        while i+z[i]<n and s[z[i]]==s[i+z[i]]:
            z[i]+=1
        if i+z[i]-1>r: l,r=i,i+z[i]-1
    return z
# 匹配 p in text: z_function(p+'#+text), z[i]>=len(p) 为匹配。
```

最长回文子串 DP 小数据

```
n=len(s); dp=[[False]*n for _ in range(n)]
best=(0,1)
for length in range(1,n+1):
    for l in range(n-length+1):
        r=l+length-1
        if s[l]==s[r] and (length<=2 or dp[l+1][r-1]):
            dp[l][r]=True
            if length>best[1]: best=(l,length)
print(s[best[0]:best[0]+best[1]])
# O(n^2), n小用; n大用 Manacher。
```

二进制子串滚动整数

```
# 是否包含所有长度 k 的二进制串
need=1<<k
if len(s)-k+1 < need: return False
seen=set(); mask=0; full=need-1
for i,ch in enumerate(s):
    mask=((mask<<1)&full) | (ord(ch)-48)
    if i>=k-1:
        seen.add(mask)
        if len(seen)==need: return True
return False
```

字符串排序构造最小拼接

```
from functools import cmp_to_key
def cmp(a,b):
    if a+b < b+a: return -1
    if a+b > b+a: return 1
    return 0
arr.sort(key=cmp_to_key(cmp))
ans=''.join(arr)
# 最大拼接反过来。
```

27. P27 DP 补充: LCS / 编辑距离 / 区间DP / 计数DP

LCS 最长公共子序列

```
def lcs(a,b):
    n,m=len(a),len(b)
    dp=[[0]*(m+1) for _ in range(n+1)]
    for i in range(n):
        for j in range(m):
            if a[i]==b[j]: dp[i+1][j+1]=dp[i][j]+1
            else: dp[i+1][j+1]=max(dp[i][j+1],dp[i+1][j])
    return dp[n][m]
```

编辑距离

```
def edit(a,b):
    n,m=len(a),len(b)
    dp=[[0]*(m+1) for _ in range(n+1)]
    for i in range(n+1): dp[i][0]=i
    for j in range(m+1): dp[0][j]=j
    for i in range(1,n+1):
        for j in range(1,m+1):
            dp[i][j]=min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+(a[i-1]!=b[j-1]))
    return dp[n][m]
```


零钱兑换: 最少硬币

```
dp=[INF]*(amount+1); dp[0]=0
for x in coins:
    for s in range(x, amount+1):
        dp[s]=min(dp[s], dp[s-x]+1)
print(-1 if dp[amount]==INF else dp[amount])
```

路径计数网格

```
dp=[[0]*m for _ in range(n)]
dp[0][0]=1
for i in range(n):
    for j in range(m):
        if grid[i][j]=='#': dp[i][j]=0; continue
        if i: dp[i][j]+=dp[i-1][j]
        if j: dp[i][j]+=dp[i][j-1]
# 可取模。
```

区间 DP 框架

```
# dp[l][r] 由更短区间转移
for length in range(1,n+1):
    for l in range(n-length+1):
        r=l+length-1
        dp[l][r]=base
        for k in range(l,r):
            dp[l][r]=min(dp[l][r], dp[l][k]+dp[k+1][r]+cost(l,k,r))
# 石子合并/矩阵链。
```

树上背包提示

树形 DP 若每个节点有容量/选择数, 通常 dfs 子树后合并孩子 dp; 复杂度常 $O(nW^2)$, 先看数据范围。

28. P28 矩阵/模拟/几何小工具

方向与转向

```
dirs=[(0,1),(1,0),(0,-1),(-1,0)] # 右下左上
# 右转: d=(d+1)%4; 左转: d=(d+3)%4; 掉头: d^=2
x+=dirs[d][0]; y+=dirs[d][1]
```

矩阵旋转/转置

```
# 顺时针旋转 90
rot = [list(row) for row in zip(*mat[::-1])]
# 逆时针
rot = [list(row) for row in zip(*mat)][::-1]
# 转置
trans = [list(row) for row in zip(*mat)]
```

螺旋遍历

```
top,bottom,left,right=0,n-1,0,m-1; ans=[]
while top<=bottom and left<=right:
    for j in range(left,right+1): ans.append(a[top][j])
    top+=1
    for i in range(top,bottom+1): ans.append(a[i][right])
    right-=1
    if top<=bottom:
        for j in range(right,left-1,-1): ans.append(a[bottom][j])
        bottom-=1
    if left<=right:
        for i in range(bottom,top-1,-1): ans.append(a[i][left])
        left+=1
```

坐标/曼哈顿

```
manhattan = abs(x1-x2)+abs(y1-y2)
# 八方向 Chebyshev 距离: max(abs(dx),abs(dy))
# 欧氏平方: dx*dx+dy*dy, 比较距离不必开根。
```

扫描线差分思想

```
# 区间覆盖数量最大: 端点事件
events=[]
for l,r in intervals:
    events.append((l,1)); events.append((r,-1)) # 若 [l,r) 这样写
cur=best=0
for x,t in sorted(events):
    cur+=t; best=max(best,cur)
```

模拟题防炸

把动作拆函数: inside(), move(), turn(), apply_event()。状态变量统一放一起。每步复杂度若太高, 找周期/取模/队列优化。

29. P29 数学/数论/组合补充

埃氏筛

```
def sieve(n):
    isprime=[True]*(n+1); isprime[0:2]=[False,False]
    for i in range(2,int(n**0.5)+1):
        if isprime[i]:
            for j in range(i*i,n+1,i): isprime[j]=False
    return [i for i in range(n+1) if isprime[i]]
```

质因数分解

```
def factorize(x):
    res=[]; d=2
    while d*d<=x:
        if x%d==0:
            c=0
            while x%d==0: x//=d; c+=1
            res.append((d,c))
        d+=1
    if x>1: res.append((x,1))
    return res
```

扩展欧几里得/逆元

```
def exgcd(a,b):
    if b==0: return a,1,0
    g,x1,y1=exgcd(b,a%b)
    return g,y1,x1-a//b*y1
# a 在 mod 下逆元存在 iff gcd(a,mod)=1
g,x,y=exgcd(a,mod); inv=x%mod
# 若 mod 为质数: inv=pow(a,mod-2,mod)
```

组合数取模预处理

```
MOD=10**9+7; N=200000
fac=[1]*(N+1); invfac=[1]*(N+1)
for i in range(1,N+1): fac[i]=fac[i-1]*i%MOD
invfac[N]=pow(fac[N],MOD-2,MOD)
for i in range(N,0,-1): invfac[i-1]=invfac[i]*i%MOD
def C(n,k):
    if k<0 or k>n: return 0
    return fac[n]*invfac[k]%MOD*invfac[n-k]%MOD
```

取模注意

$(a+b)\%MOD$, $(a-b)\%MOD$, $a*b\%MOD$ 。

除法不能直接 $/$ ，要乘逆元。Python 的负数 $\% MOD$ 结果非负，通常方便。

30. P30 数据结构补充: 懒删除堆 / 双堆中位数 / Ordered技巧

懒删除堆思想

```
# heapq 不支持  $O(\log n)$  删除任意元素。做法: Counter 记录待删除, 堆顶失效就弹。
delcnt=Counter()
def prune(h):
    while h and delcnt[h[0]]:
        delcnt[h[0]]-=1; heapq.heappop(h)
# 大根堆存负数时 delcnt 也用负数键。
```

双堆维护中位数

```
small=[] # 大根堆, 存负
large=[] # 小根堆
def rebalance():
    if len(small)>len(large)+1:
        heapq.heappush(large, -heapq.heappop(small))
    elif len(large)>len(small):
        heapq.heappush(small, -heapq.heappop(large))
def add(x):
    if not small or x<=-small[0]: heapq.heappush(small, -x)
    else: heapq.heappush(large, x)
    rebalance()
def median():
    if len(small)>len(large): return -small[0]
    return (-small[0]+large[0])/2
```

bisect 模拟有序表

```
# 插入/删除是  $O(n)$ , n不大可用
from bisect import bisect_left, insort
arr=[]
insort(arr, x)
i=bisect_left(arr, x)
if i<len(arr) and arr[i]==x: arr.pop(i)
```

Counter 多重集合

```
cnt=Counter()
cnt[x]+=1
cnt[x]-=1
if cnt[x]==0: del cnt[x]
# 两个 Counter 可做交并: c1 & c2, c1 | c2
```

defaultdict 嵌套

```
g=defaultdict(list)
mp=defaultdict(lambda: defaultdict(int))
# 但考试中别过度炫技, 普通 dict 更好 debug。
```

队列模拟

```
# 固定窗口/缓冲区/打印机/约瑟夫
q=deque(range(n))
while len(q)>1:
    q.rotate(-(k-1))
    q.popleft()
print(q[0])
```

31. P31 经典题型更细定位

最短桥/孤岛距离

1 DFS/BFS 找第一座岛, 把所有岛格入队, dist=0。2 多源 BFS 向海扩张。3 第一次碰到第二座岛时距离即答案。

八皇后/排列回溯

```
cols=set(); diag1=set(); diag2=set()
def dfs(r):
    if r==n: ans+=1; return
    for c in range(n):
        if c in cols or r+c in diag1 or r-c in diag2: continue
        cols.add(c); diag1.add(r+c); diag2.add(r-c)
        dfs(r+1)
        cols.remove(c); diag1.remove(r+c); diag2.remove(r-c)
```

迷宫钥匙门状态

状态必须包含钥匙集合: (x,y,mask)。遇到钥匙更新 mask; 遇到门检查 mask 是否有对应位。BFS 求最短步数。

最少操作变换数字

每个数字是状态，操作生成邻居，BFS。若范围大，双向 BFS。注意 visited 防环。

括号生成/合法序列

```
def gen(n, l=0, r=0, s=''):
    if len(s)==2*n: ans.append(s); return
    if l<n: gen(n, l+1, r, s+'(')
    if r<l: gen(n, l, r+1, s+')')
```

排序后双指针三数和

```
a.sort(); res=[]
for i in range(n-2):
    if i and a[i]==a[i-1]: continue
    l, r=i+1, n-1
    while l<r:
        s=a[i]+a[l]+a[r]
        if s==0:
            res.append((a[i], a[l], a[r])); l+=1; r-=1
            while l<r and a[l]==a[l-1]: l+=1
            while l<r and a[r]==a[r+1]: r-=1
        elif s<0: l+=1
        else: r-=1
```

树序列合法性

BST 后序/前序合法性常用递归区间上下界；完全二叉树数组下标: 左 $2i+1$, 右 $2i+2$ ；堆性质: $\text{parent} \leq \text{children}$ 小根堆。

32. P32 最终总索引 + 取舍策略

按关键词快速定位

最短路: P8/P22/P23；欧拉/单词链: P10/P24；强连通/桥/割点: P10/P23；MST/DSU: P9；树遍历建树: P11-P13；表达式: P4/P12；字符串: P15/P26；DP: P16/P27；启发式/状态搜索: P6/P22；库函数: P2/P21；Bug: P19。

考场优先级

第一优先: 输入输出、栈队列、DFS/BFS、树遍历建树、heapq、DSU、Dijkstra、拓扑、MST、KMP、排序二分。

第二优先: Floyd、Bellman-Ford、Tarjan、Manacher、DP模板、AIDA。

最后看: Dinic、线段树、BIT、AVL、倍增。

不会写时的保底

1 写暴力拿部分分。2 用内置库先过小数据。3 若图权复杂，先 BFS/DFS 判连通。4 若输出路径复杂，先输出长度/可行性调通。5 卡住就换题，别和一个模板死磕。

复杂度保底判断

$n \leq 20$: 2^n /回溯/状压。 $n \leq 500$: $O(n^3)$ 可能。 $n \leq 3000$: $O(n^2)$ 勉强。 $n \leq 1e5$: $O(n \log n)$ 。图: $V+E$ 可; $E \log V$ 可; V^3 只适合小 V 。

缩印打印建议

本版是 32 个逻辑页。若允许 10 张 A4 正反，可打印 2 页/面 + 双面，则 32 页只占 8 张 A4。建议横向、双面长边翻转、2 pages per side; 字号看不清就改 1 页/面打印最常用 P1-P20。

最终提醒

模板不是越高级越好。先识别题型，再选最短可靠模板。Python3.8 记得 `@lru_cache(None)`，别写 `@cache`; BFS 用 `deque`; 堆用 `heapq`; 递归先 `setrecursionlimit`。